

Project 2: 图像有损信源编码传输

曾闻天 231880147

1 实验目标

项目二要求实现一个针对图像的有损源编码以及基于二进制信道失真（BSC）和基于二进制对称信道失真（BEC）的简单信道编码。

实验结果从 PSNR、像素准确度、算法复杂度和比特误码率（BER）等维度对系统性能进行综合分析。

2 实验过程

2.1 图像读取与预处理

函数: `load_gray_image(path: Path, max_side: int = 1280) -> np.ndarray`

使用 PIL 库将图像转换为灰度模式，若图像长边超过 `max_side`（默认 1280 像素），则按比例缩放至该尺寸以内，缩放采用 LANCZOS 重采样以保证质量。

最终返回 `uint8` 类型的 numpy 数组，形状为 `(height, width)`。

2.2 DCT 变换与逆变换

函数: `dct2(block: np.ndarray) -> np.ndarray`

函数: `idct2(block: np.ndarray) -> np.ndarray`

实现 8×8 块的二维离散余弦变换。首先通过 `_dct8_matrix` 函数预先计算 DCT 变换矩阵 C ，其中：

$$C_{k,i} = \begin{cases} \frac{1}{\sqrt{8}} & k = 0 \\ \sqrt{\frac{2}{8}} \cos\left(\frac{\pi k(2i+1)}{16}\right) & k > 0 \end{cases}$$

正变换为 $C \cdot block \cdot C^T$ ，逆变换为 $C^T \cdot block \cdot C$ 。

2.3 量化表生成

函数: `scale_quant_table(quality: int) -> np.ndarray`

基于 JPEG 标准亮度量化表，根据质量因子 q (1-100) 动态缩放。缩放规则为：

$$\text{scale} = \begin{cases} 5000/q & q < 50 \\ 200 - 2q & q \geq 50 \end{cases}$$
$$Q_{\text{scaled}}(u, v) = \max \left(1, \left\lfloor \frac{\text{JPEG_LUMA}(u, v) \times \text{scale} + 50}{100} \right\rfloor \right)$$

2.4 Zigzag 扫描与重排

函数: `block_to_zigzag(qblock: np.ndarray) -> List[int]`

函数: `zigzag_to_block(coeffs: Sequence[int]) -> np.ndarray`

将 8×8 的 DCT 系数矩阵按 Zigzag 顺序展开为一维数组。该顺序优先保留低频系数，使高频零值连续出现，便于游程编码。预定义的 ZIGZAG 列表存储了 64 个坐标对 (r, c) ，按此顺序遍历即可完成映射，逆变换同理。

2.5 游程编码 (RLE) 与 DPCM

函数: `rle_encode(coeffs: Sequence[int]) -> List[Tuple[int, int]]`

对 Zigzag 后的 64 个系数（索引 0 为 DC，索引 1-63 为 AC）进行编码：

- DC 系数：采用差分脉冲编码调制 (DPCM)，编码值为 $DC_{\text{diff}} = DC_{\text{curr}} - DC_{\text{prev}}$ ，利用相邻块的 DC 相关性进一步压缩。
- AC 系数：游程编码，将序列编码为（连续零个数，非零值）对。连续 16 个零时编码为 (15, 0)，序列末尾以 (0, 0) 作为 EOB (End of Block) 标记。

函数: `rle_decode(runs: Sequence[Tuple[int, int]], ac_len: int = 63) -> List[int]`

解码时按游程对展开，遇到 EOB 后剩余位置补零。

2.6 编码与解码

函数: `encode_grayscale(arr: np.ndarray, quality: int) -> bytes`

首先将图像的高度和宽度分别填充到 8 的倍数，填充方式采用边缘复制填充，以确保图像能够被完整划分为 8×8 的像素块。

接下来，遍历每一个 8×8 块，将每个像素值减去 128 使其中心归零，然后执行 DCT。变换后的系数按照预先定义的量化表进行量化，并四舍五入取整，从而实现有损压缩的核心步骤。

量化完成后，对每个块进行 Zigzag 扫描，将二维系数矩阵转换为一维序列。其中，DC 系数采用 DPCM；而 AC 系数则采用 RLE 来记录连续零值的个数与非零系数的值。每个块编码完成后，将得到的 DPCM 差值和 RLE 结果存入列表。

最后，将图像的原始尺寸、所使用的质量因子以及所有块的编码数据一起序列化为 JSON 格式，再使用 zlib 以最高压缩级别进行压缩，得到最终的压缩文件。

函数：decode_grayscale(data: bytes) -> np.ndarray

解码为逆过程：解压 JSON，逐块读取 DPCM 值恢复 DC，RLE 解码恢复 AC，Zigzag 重排回矩阵，乘以量化表后执行 IDCT，最后裁剪填充区域并截断到 0-255 范围。

2.7 比特流与字节流转换

函数：bytes_to_bits(data: bytes) -> str

函数：bits_to_bytes(bits: str) -> bytes

bytes_to_bits 将每个字节转换为 8 位二进制字符串并拼接。bits_to_bytes 将二进制字符串补 0 至 8 的倍数，然后每 8 位转换为一个字节。

2.8 Hamming (7,4) 编解码

函数：hamming74_encode_nibble(d: Sequence[int]) -> List[int]

输入 4 个数据比特 $[d_0, d_1, d_2, d_3]$ ，生成 3 个校验比特：

$$p_1 = d_0 \oplus d_1 \oplus d_3$$

$$p_2 = d_0 \oplus d_2 \oplus d_3$$

$$p_3 = d_1 \oplus d_2 \oplus d_3$$

输出码字 $[p_1, p_2, d_0, p_3, d_1, d_2, d_3]$ 。

函数：hamming74_decode_codeword(r: List[int]) -> Tuple[List[int], int]

译码时计算伴随式：

$$s_1 = p_1 \oplus d_0 \oplus d_1 \oplus d_3$$

$$s_2 = p_2 \oplus d_0 \oplus d_2 \oplus d_3$$

$$s_3 = p_3 \oplus d_1 \oplus d_2 \oplus d_3$$

伴随式 (s_1, s_2, s_3) 的二进制值对应错误位置 (1-7)，将该位置取反即完成单比特纠错。

函数：hamming_encode_bits(bits: str) -> str

函数：hamming_decode_bits(bits: str) -> Tuple[str, int]

将输入比特流按 4 位分组，每组编码为 7 位码字后拼接。解码时按 7 位分组译码，返回数据比特流和累计纠正次数。

2.9 重复码编解码

本实验中设定每个信息比特重复的次数为常量 $\text{REP_FACTOR} = 10$ 。

函数: `repetition_encode(bits: str, n: int = 5) -> str`。

其逻辑是：将输入比特串中的每一个比特，按照参数 n （默认为 10）重复相应的次数，从而得到编码后的比特串。

函数: `repetition_decode_hard(bits: str, n: int = 10) -> str`

适用于 BSC 场景下的硬判决译码，即将接收到的比特串按每 n 比特划分为一组，对每一组分别统计其中 0 和 1 出现的次数，取出现次数较多的那个比特作为判决输出。

函数: `repetition_decode_erasure(bits: str, n: int = 10) -> str`

适用于 BEC 场景下的擦除译码，即在每组 n 个比特中，忽略被标记为擦除 (E) 的符号位置，仅对剩余未擦除的已知比特进行多数投票。如果某一组中所有比特都被擦除了，即没有任何已知信息可供判决，则默认将该位解码为 0。

2.10 信道模型

函数: `bsc_channel(bits: str, p: float, rng: random.Random) -> Tuple[str, int]`

BSC 信道：对输入比特串的每一位，以概率 p 翻转，以概率 $1 - p$ 保持不变。返回输出比特串和翻转次数。

函数: `bec_channel(bits: str, epsilon: float, rng: random.Random) -> Tuple[str, int]`

BEC 信道：对每一位，以概率 ε 输出 E，以概率 $1 - \varepsilon$ 输出原比特。返回输出字符串和擦除次数。

2.11 BEC 下 Hamming 译码的擦除处理

函数: `hamming_decode_erasure_bits(bits: str) -> Tuple[str, int]`

当 BEC 产生擦除后，接收到的比特串可能含有 E。本函数采用穷举搜索方法：对每个 Hamming (7,4) 码字中所有擦除位置，尝试 0/1 两种填充，检查是否满足三个校验方程。若存在唯一合法解，则完成纠错；若无解，则默认填充 0。返回解码后的比特串和解析的擦除符号数。

2.12 码流封装与解封装

函数: `pack_bitstream(source_payload: bytes) -> bytes`

封装格式：

[4B 魔数"IMG2"] + [4B 源载荷长度] + [4B 原始比特长度] + [Hamming 编码后的比特流 (字节对齐)]

函数:unpack_bitstream(data: bytes, channel_bits: str, channel: str) -> Tuple[bytes, int]

根据信道类型选择对应的 Hamming 译码器（普通或擦除版本），然后截取有效比特长度，转换回字节流，返回源载荷和纠正次数。

2.13 端到端

函数:run_single_image(path: Path, channel: str, p_bsc: float, eps_bec: float, quality: int, seed: int) -> PipelineResult

对于单张灰度图像，首先加载待处理的灰度图像，作为原始信源。随后，对图像执行有损信源编码，并将编码结果封装为便于传输的格式，最终输出为比特流。

在信道编码阶段，先对比特流应用 Hamming (7,4) 编码，再叠加重复码 (10,1)，以增强传输过程中的抗差错能力。编码后的数据随后通过信道进行传输。信道类型可以根据实验需求选择理想信道、BSC 信道或 BEC 信道，以模拟不同的通信环境。

在接收端，首先根据信道类型选择相应的重复码译码方式：如果是 BSC 信道，采用硬判决译码；如果是 BEC 信道，则采用擦除译码。完成重复码译码后，再进行 Hamming 译码，同样根据信道类型选择普通译码或支持擦除的版本。

译码恢复出的比特流经过解封装操作，还原为有损信源编码的中间格式，然后执行有损信源解码，最终重建出图像。为了评估整个系统的性能，在流程结束后会计算多个指标，包括 PSNR、像素准确度、BER，以及各个处理阶段所消耗的时间。

2.14 性能评估函数

函数:psnr(original: np.ndarray, restored: np.ndarray) -> float

计算 PSNR:

$$\text{MSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (x_{ij} - y_{ij})^2, \quad \text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \text{ dB}$$

函数:pixel_accuracy(original: np.ndarray, restored: np.ndarray, tol: int = 0) -> float

统计 $|x_{ij} - y_{ij}| \leq \text{tol}$ 的像素占比，实验中 tol 取 1，允许 ± 1 灰度误差。

函数:bit_error_rate(sent: str, received: str) -> float

计算 BER，注意忽略接收串中的 E 符号，仅比较已知比特位。

2.15 报告输出

函数:print_result(r: PipelineResult) -> None:

输出内容包括:

指标	说明
信道模型	理想信道、BSC、BEC
质量因子	40、75、90
PSNR	峰值信噪比
准确度	像素准确度
BER	码流
信道错误数	信道的破坏程度
Hamming 纠正	纠错能力
耗时	衡量效率

3 交互

3.1 交互菜单 (interactive_loop)

支持三种操作：

1. 运行完整实验 (IDEAL + BSC + BEC, 打印报告)
2. 单张图像编解码 (保存还原 PNG)
3. 查看最近一次实验 JSON

4 实验结果

1. 运行 pro2.bat, 选择对 images 文件夹中的 test1.jpg、test2.jpg 运行完整实验。

2. 程序输出：

汇总报告 (Project 2 - 有损信源编码 + 信道编码)						
图像	信道	质量	PSNR	准确度	BER	ms
test1	IDEAL	40	33.67	43.21	0.0000	3363.4
test1	IDEAL	75	37.19	55.61	0.0000	5010.0
test1	IDEAL	90	41.56	68.29	0.0000	7702.8
test1	BSC	40	33.67	43.21	0.0000	5684.3
test1	BSC	75	37.19	55.61	0.0000	6667.0
test1	BSC	90	41.56	68.29	0.0000	11071.1
test1	BEC	40	33.67	43.21	0.0000	5029.1
test1	BEC	75	37.19	55.61	0.0000	7914.6
test1	BEC	90	41.56	68.29	0.0000	12955.4
test2	IDEAL	40	31.26	30.97	0.0000	3374.7
test2	IDEAL	75	34.61	39.65	0.0000	5463.5
test2	IDEAL	90	39.14	51.67	0.0000	8557.4
test2	BSC	40	31.26	30.97	0.0000	4539.1
test2	BSC	75	34.61	39.65	0.0000	7542.2
test2	BSC	90	39.14	51.67	0.0001	12471.7
test2	BEC	40	31.26	30.97	0.0000	5457.1
test2	BEC	75	34.61	39.65	0.0000	8917.7
test2	BEC	90	39.14	51.67	0.0000	14662.7
平均 PSNR: 36.24 dB 平均像素准确度: 48.23% 平均耗时: 7576.9 ms						

图 1: test2

3. 结果分析

在本次有损信源编码加信道编码的联合实验中，应用 Hamming (7,4) 编码，再叠加重复码 (10,1) 的信道编码方案，分别在理想信道、BSC 信道和 BEC 信道下对两幅测试图像进行传输。

实验结果表明，所有信道条件下的 BER 均为 0，说明所采用的联合纠错编码能够完全纠正 BSC 和 BEC 引入的信道错误，实现了无差错的传输。

同时，随着质量因子从 40 提高到 90，重建图像的 PSNR 和像素准确度均显著提升，验证了有损信源编码率失真性能的合理性。

从运行时间来看，BEC 因采用遍历补齐擦除位的译码方式，计算复杂度最高，耗时明显超过 BSC 和理想信道。

总体而言，重复码与汉明码的级联方案在中等信道错误率条件下表现出极强的纠错能力，但代价是较低的码率和较高的译码延迟，尤其在擦除信道下更为突出。

5 总结

通过上述实验，项目二的研究目标基本实现。